

同济大学课程考核试卷（A 卷）

2023—2024 学年第一学期

命题教师签名：

审核教师签名：

课号：101020

课名：操作系统

考试考查：考试

此卷选为：期中考试()、期终考试(☒)、重考()试卷

年级_____专业_____学号_____姓名_____得分_____

一、（6 分）概念题

1、普通磁盘文件，读操作是同步的，写操作是异步的。为什么。

答：读操作是同步的。这是因为进程要处理从磁盘读入的文件数据，继续运行之前必须同步等待 IO 操作结束，内存中有可供处理的文件数据副本。（1.5 分）

写操作是异步的。这是因为进程产生新数据后，写操作分两步（1）新数据写入缓存块（2）启动 IO 操作，DMA 控制器将缓存块写入磁盘。第 1 步完成后，进程无需同步等待 IO 操作结束，继续执行与写操作完成与否无关，所以是异步的。（1.5 分）

2、磁盘数据块为什么要先读后写。在什么情况下不需要先读后写。

答：磁盘写操作以数据块为单位，会用内存中的整块数据覆盖磁盘数据块。所以，正确的操作步骤应该是（1）先读，确保缓存块中包含磁盘数据块中的旧数据（2）后写，将新数据存入缓存块（3）硬件（DMA 控制器和磁盘控制器）将既包含新数据，又包含旧数据的缓存块写回磁盘。

如果没有先读操作，内存中的错误信息会覆盖磁盘数据块中的旧数据，导致文件内容出错。（1.5 分）

数据块写满，无需先读后写。（1.5 分）

二、（12 分）系统调用不同于一般的子程序调用。请问：UNIX V6++系统调用如何：

1、传递应用程序想要执行的系统调用号？ 用 EAX 寄存器 （3 分）

2、传递系统调用的参数？ 用 EBX, ECX, EDX, ESI, EDI 寄存器 （3 分）

3、将系统调用的返回结果传给应用程序？ 系统调用的返回值用 EAX 寄存器。其它数据，可以存入其它通用寄存器 或是 钩子函数传入的指针变量所指向的用户空间内存区域。（3 分）

4、传统的子程序调用如何传参，如何传递返回值？ 压栈传参（2 分）。 用 EAX 寄存器传递返回值（1 分）。

三、（8 分）Unix V6++系统，父进程创建子进程使用 fork 系统调用（1 分）？简述该系统调用的执行过程。

答：父进程执行 fork 系统调用的钩子函数陷入核心态，调用 Newproc() 函数创建子进程。

Newproc()函数 (1) 为子进程分配 Process 结构和 pid, 复制父进程的 Process 结构 (2 分) (2) 为子进程分配相对表, 复制父进程的相对表 (1 分) (3) 子进程复制父进程的用户空间 (3.1) 共享父进程的正文段 (3.2) 分配可执行存储器, 复制父进程的可交换部分 (2 分)。完成后 (1) 父进程 fork 系统调用返回, 返回值是子进程的 pid 号 (2) 子进程就绪, 选中成为现运行进程后, fork 系统调用返回, 返回值是 0, 若图像在盘交换区, 需要等待 0#进程为可交换部分分配内存 (2 分)。
相对表。。。放在盘交换区, 扣 1 分

四、(5 分) 某机械硬盘, 磁头当前位于磁道 143。若磁道访问请求序列为: 86, 147, 91, 177, 94, 150, 102, 175, 130。按照电梯调度方法处理完上述请求后, 请 (5 分)

(1) 写出磁头移动轨迹

143 → 147 → 150 → 175 → 177 → 130 → 102 → 94 → 91 → 86

(2) 计算磁头移动的总磁道数。

$(177-143) + (177-86) = 125$ 。计算错误, 扣 1 分

五、(8 分) 全嵌套中断处理模式。低优先级中断处理程序运行时, 系统响应高优先级中断处理请求。已知, 时钟中断优先级高于磁盘中断优先级。系统并发 3 个进程: PX、PA 和 PB。

- 890s, PA 进程执行系统调用 sleep (10) 入睡, 入睡优先数是 90。
- 899s, PB 进程执行 read 系统调用读磁盘文件, 入睡优先数是-50。
- 900s, 现运行进程 PX 正在执行应用程序。PA 设置的闹钟到期、PB 读取的磁盘文件数据 IO 结束。系统先响应时钟中断。

试分析 900s, 系统详细的调度过程。

答: 现运行进程 PX 执行时钟中断处理程序, 更新 Time::time, STI、EOI 后, 中断嵌套, 响应磁盘中断。

磁盘中断处理程序唤醒 PB、RunRun++。先前核心态, 不调度, 磁盘中断处理程序运行结束后, PX 返回、继续执行被打断的时钟中断处理程序。

时钟中断处理程序唤醒 PA、RunRun++(值是 2)。先前用户态, 例行调度。RunRun 非 0, 现运行进程 PX 被剥夺、放弃 CPU。SRUN 进程集合 { PX, 优先数 ≥ 100 ; PA, 优先数=90; PB, 优先数=-50 }。

PB 优先级最高, 被选中, 执行 read 系统调用后半段。完成后恢复其应用程序优先数 ($p_pri \geq 100$), 让出 CPU。SRUN 进程集合 { PX, 优先数 ≥ 100 ; PA, 优先数=90; PB, 优先数 ≥ 100 }。

PA 优先级最高, 被选中, 执行 sleep 系统调用后半段。waketime 到期, PA sleep 系统调用返回, 恢复应用程序优先数之后, 让出 CPU。

SRUN 进程集合 { PX, 优先数 ≥ 100 ; PA, 优先数 ≥ 100 ; PB, 优先数 ≥ 100 }。PA、PB、PX 轮流使用 CPU 执行应用程序。

中断嵌套, 唤醒 2 个进程, PB 先执行, PA 后执行 (每个 2 分, 酌情)
轮流执行应用程序

六、(10 分) 某 Unix V6++系统, 时钟中断每秒钟发生 60 次, 调度魔数 SCHMAG 是 20。现运行进程和被剥夺进程优先数计算公式: $p_pri = \min \{ 127, \text{进程的静态优先数} + (p_cpu/16) \}$ 。T 时刻整

数秒，并发 3 个 CPU 密集型进程 PA、PB 和 PC，它们只计算，不执行系统调用，不启动 IO 操作。这 3 个进程静态优先数都等于 100，PCB 分别是 Process[5]、[7]、[2]，PA 先运行。

1、 填表格（2 分）

p_cpu	T	T+1	T+2	T+3	T+4	T+5		p_pri	T	T+1	T+2	T+3	T+4	T+5	
PA	0	40	20	0	40	20		PA	100	102	101	100	102	101	
PB	0	0	40	20	0	40		PB	100	100	102	101	100	102	
PC	0	0	0	40	20	0		PC	100	100	100	102	101	100	

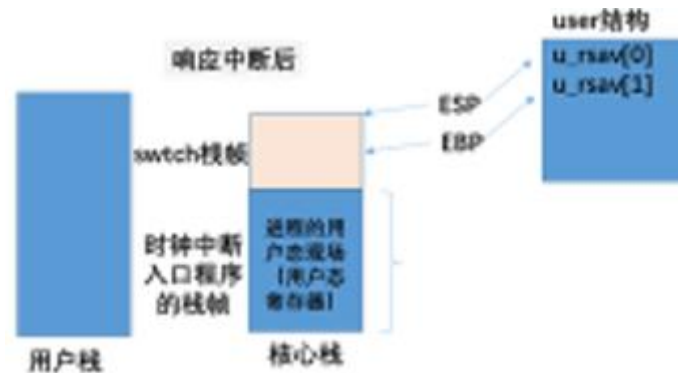
2、 T+1 时刻，PA 进程用完时间片放弃 CPU，

用户态寄存器压栈，Swch 栈帧保存进 user。少一个扣 2 分。

（1）简述 T+1 时刻 PA 怎样保护应用程序的执行现场。

PA 响应时钟中断。中断隐指令和中断入口程序将用户态寄存器压入自己的核心栈，这便保护了应用程序的执行现场。

时钟中断返回用户态前，例行调度，PA 执行 Swch()，放弃 CPU 前将 Swch 栈帧压在中断栈帧的上面，并将其基地址和栈帧顶部存入自己的 user 结构，如图。



（2）何时 PA 进程会再次得到运行机会？该时刻系统如何恢复它的应用程序执行现场（5 分）

T+3 时刻，PC 放弃 CPU 后。

Swch 函数选中 PA，找到它的 user 结构（1）恢复 Swch 栈帧（ESP 和 EBP），（2）用相对表刷新用户页表、恢复用户空间地址映射关系。Swch 函数返回，Swch 栈帧出栈，中断栈帧得以恢复。

随后，PA 从时钟中断栈帧中弹出压栈保存的用户态寄存器，恢复 CPU 执行现场。

IRET 回用户态，继续执行应用程序。

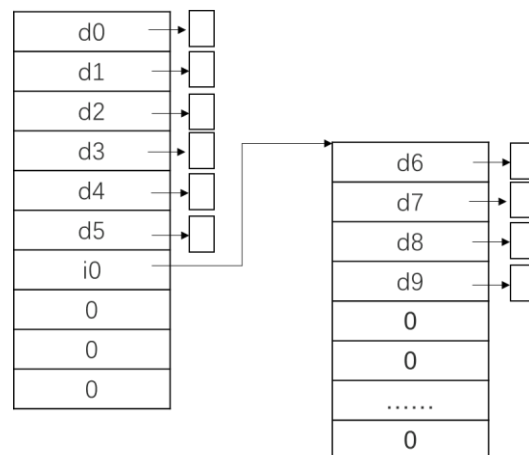
3、 当前时刻，所有进程轮转 1 轮，需要 3 秒？100s 之后，系统又新建了 3 个 CPU 密集型进程 PX、PY 和 PZ，所有进程轮转 1 轮，需要 6 秒？假设 PX、PY 和 PZ 独占使用 CPU 时，完成所有运算任务，各自需要执行 10s，请估算在题干描述的状态下，进程 PX 的周转时间 60s，精度 10s（3 分）
精确值是 58s（57.5s）都是正确答案。

七、（10 分）Unix V6++ 系统，磁盘数据块 512 字节，文件的 d_addr 数组管理 6 个直接块，2 个一次索引块和 2 个二次索引块。本题读入指磁盘 IO，假定打开时 fileA 所有数据块缓存不命中。请问：

（1）系统需要为一个长度为 5120 字节的文件 fileA 分配多少磁盘存储资源？（2 分）

答：一个目录项，一个 DiskInode（1 分），10 个数据块和 1 个索引块。（1 分）

(2) 填写 fileA 的 d_addr 数组并画出其混合索引结构，分配给该文件的磁盘数据块号（扇区号）可以自定。（2 分）



(3) 现运行进程成功打开该文件后，访问偏移量是 0 的字节需要读入1个磁盘数据块。访问偏移量是 3600 的字节需要读入2个磁盘数据块，随后访问偏移量是 3601 的字节需要读入0个磁盘数据块，访问偏移量是 4096 的字节需要读入1个磁盘数据块。（4 分）

(4) 简述混合索引这种文件物理结构的优缺点。（2 分）

优点（1）索引结构灵活、扩充方便：小文件无需索引块，大型、巨型文件索引块随文件长度增加逐步分配，无需改动已有结构（2）支持随机访问，工作效率高，访问文件首部 6 个数据块无需索引块，速度快。

缺点：对插入、删除操作不友好。文件内部插入、删除数据块，索引结构改动非常大。

酌情

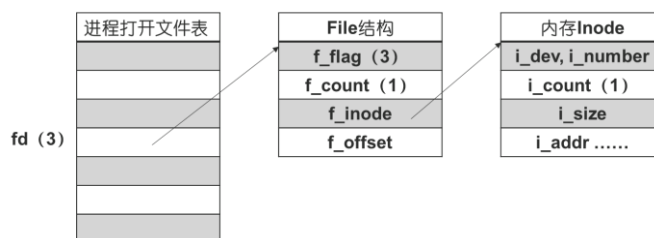
八、（14 分）假如文件 abc 大小为 500 字节。程序运行时需要引用的所有磁盘数据缓存不命中。请问

```
main()
{
    char data[300];
    int fd = open("abc", O_RDWR); //以可读可写方式打开文件
    int count = read ( fd, data, 300);
    seek(fd, 500, 0);
    write ( fd, data, count);
    close(fd);
}
```

(1) 这段程序完成什么功能？ 将文件首部的 300 个字节追加写至文件末尾 （2 分）。

(2) `open` 系统调用会引发 IO 操作吗? 会。如果会, 进程会从磁盘读入 父目录文件的数据块 和 `abc` 文件的 `DiskInode` (2 分)。父目录文件的数据块不提, 不扣分。

(3) 请绘制这个进程的打开文件结构 (2 分)。



(4) `read` 系统调用的返回值为 300, 执行完毕后文件读写指针值是 300。(2 分)。**800 也对**

(5) 请简述本题代码中 `write` 系统调用的执行过程 (3 分)。

答:

`f_offset==500`, 分 2 次写入。初始化 IO 参数:

`m_offset = f_offset = 500, m_base = data, m_count = 300`

- 当前逻辑块 $500/512=0$, 块内偏移量 $500\%512=500$ 。写入 12 字节, 缓存块未写满, 需先读。缓存命中, 无 IO, `Bread` 锁住分配给 0#逻辑块的缓存块。`data[0]~data[11]`写入该缓存块。写至底部, 异步写回磁盘, 不睡。修改 IO 参数: `m_offset = 512, m_base = data+12, m_count = 288`, 非 0, 继续写。(1 分)

- 当前逻辑块 $512/512=1$, 块内偏移量 $512\%512=0$ 。混合索引表中 1#逻辑块的物理块号是 0, 系统为其分配新数据块 `new`, 登记: `i_addr[1]=new`。写入 288 字节, 未写满, 需先读, 启动 IO 将磁盘数据块 `new` 同步读入分配给它的缓存块。`data[12]~data[299]`写入该缓存块。未写至底部, 延迟写: `B_DELWR=1, B_DONE=1`, 释放。修改 IO 参数, `m_offset = 800, m_count = 0`。(1 分)

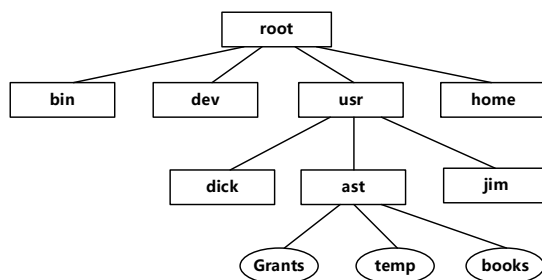
- `write` 系统调用结束, `f_offset=800`, 返回实际写入文件的字节数 300。写操作导致文件长度增加, `i_size = 800`。(1 分)

(6) 如果 `close` 系统调用执行完毕后系统断电, 文件系统会出现什么情况。应当怎样加以弥补 (3 分)。

脏数据未落盘, 写入 1#逻辑块的数据丢失。(1.5 分)

弥补的方法: 脏数据定时写回磁盘。(1.5 分)

九、(9 分) 某 UNIX V6++系统中, 文件系统的目录结构如下图所示:



(1) (3 分) 根据上述目录结构，补充完整下面几个目录文件的内容（INode 节点号请任意指定。后续问题回答中请沿用相同的 INode 号）。

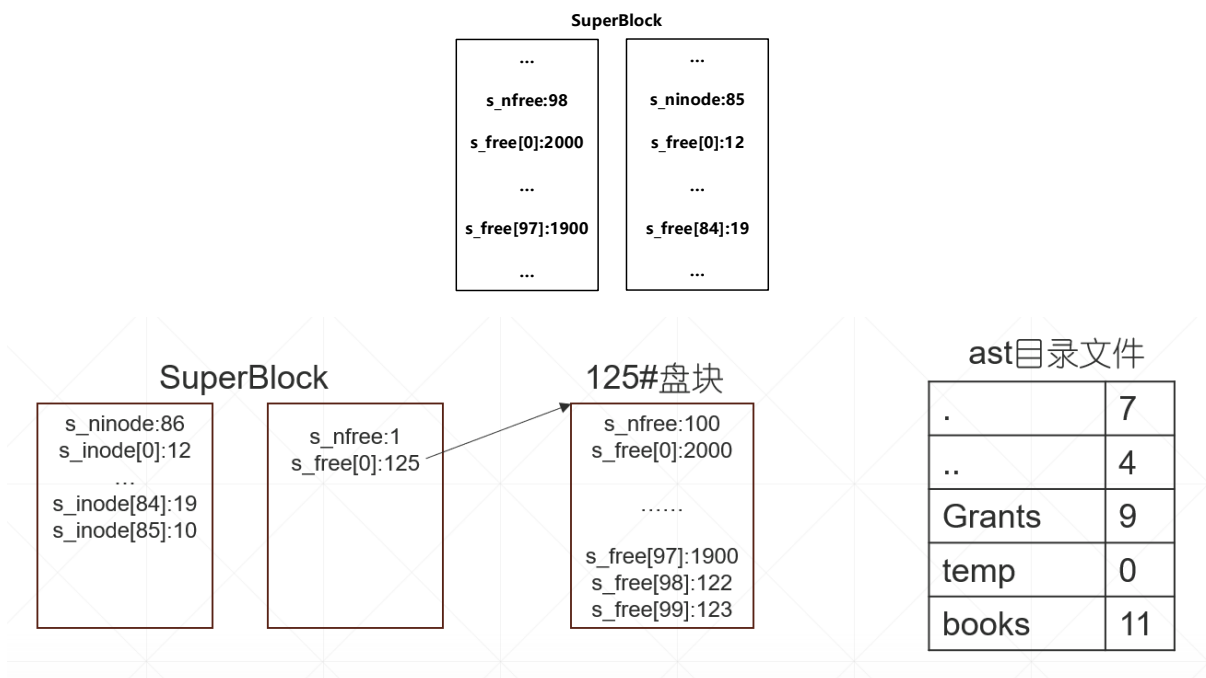
/根目录文件			usr目录文件			ast目录文件		
根目录的Inode			usr目录的Inode			ast目录的Inode		
.....	.	1		.	4		.	7
d_addr[0] = 101	..	1	d_addr[0] = 101	..	1	d_addr[0] = 101	..	4
.....	bin	2		dick	6		Grants	9
	dev	3		ast	7		temp	10
	usr	4		jim	8		books	11
	home	5						

特殊目录项，1 分

目录号和盘块号弄混了，扣 1 分

目录结构，1 分

(2) (3 分) T0 时刻，SuperBlock 中空闲 INode 栈和空闲盘块栈中记录的内容如下图所示。此时，进程 PA 删除 d_link==1 的文件/usr/ast/temp。已知，temp 文件长 1500 个字节，占用磁盘上的 122#盘块（0#逻辑块）、123#盘块（1#逻辑块）和 125#盘块（2#逻辑块）。请以图示方式说明该操作结束后，SuperBlock 中空闲 INode 栈和空闲盘块栈的变化情况。各目录文件的内容是否有变化？如果有，请图示说明。



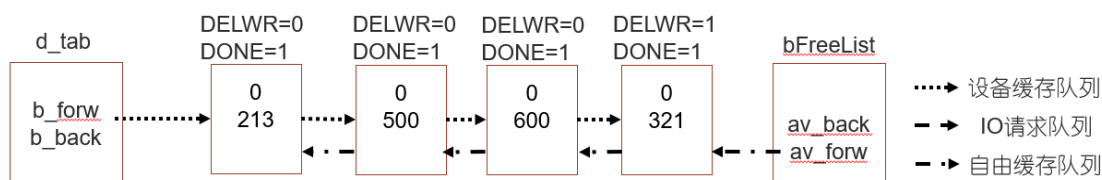
Inode 栈，空闲盘块号栈，目录文件，每个 1 分

(3) (3 分) 随后的 T1 时刻，系统创建一个新文件：/usr/ast/tempNext，并向其中写入字符串“Hello World! \0”，关闭该文件。已知，(T0,T1]时间区间，系统不曾对文件系统实施任何操作。请绘制出 ast 目录文件的变化，并给出 tempNext 文件的索引结构。



错一个，扣 1 分。

十、(6 分) T 时刻，磁盘高速缓存状态如下所示。PA、PB、PC 进程同时发起对 660#块，偏移量为 0 的字节访问请求，PA 先执行同步读操作。PB 写，PC 读。请回答以下问题。



(1) 画出读写操作全部完成后，磁盘高速缓存的状态。(2 分)

设备队列 660 (DELWR=1, DONE=1) → 213 → 500 → 321 (DELWR=0, DONE=1)

自由缓存队列 500 → 213 → 321 → 660

和这个不一样，就扣 1 分。

(2) PA 能读到 PB 写入的新数据吗？(2 分)

不可能。这是因为 660 号数据块的 0#字节，PA 先读，PB 后写。

(3) PC 能读到 PB 写入的新数据吗？(2 分)

有可能，不一定。PA 解锁数据块后，PB、PC 都有可能先运行。若 PB 先运行，0#字节 PB 先写、PC 后读，可以获得 PB 写入的新数据。PC 先运行，读不到。

没有提到不一定，扣 1 分。

十一、综合题 (12 分)

(1) 内核函数 Sleep 的功能_____现运行进程入睡，放弃 CPU_____。

内核函数 WakeUpAll 的功能_____唤醒睡眠进程_____。(1 分*2)

(2) 代码填空 (4 分)


```

void Process::Sleep(unsigned long chan, int pri)
{
    .....
    if ( pri > 0 )
    {
        ..... // 忽略信号处理
        X86Assembly::CLI();
        this->p_wchan = chan;
        this->p_stat = Process::SWAIT;
        this->p_pri = pri;
        X86Assembly::STI();

        if ( procMgr.RunIn != 0 ) {
            procMgr.RunIn = 0;
            procMgr.WakeupAll((unsigned long)&procMgr.RunIn);
        }

        Kernel::Instance().GetProcessManager().Swch( ) ;
    }
}

void ProcessManager::WakeupAll(unsigned long chan)
{
    for(int i = 0; i < ProcessManager::NPROC; i++)
    {
        if( this->process[i].p_wchan == chan)
        {
            this->process[i].SetRun();
        }
    }
}

void Process::SetRun()
{
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();

    this->p_wchan = 0;
    this->p_stat = Process::SRUN;
    if ( this->p_pri < procMgr.CurPri )
        procMgr.RunRun++;

    if ( 0 != procMgr.RunOut && (this->p_flag & Process::SLOAD) == 0 )
    {
        procMgr.RunOut = 0;
        procMgr.WakeupAll((unsigned long)&procMgr.RunOut);
    }
}

```

p_stat, 1 分。Swch, 1 分。SRUN, RunRun++, 1 分。唤醒 0#进程, 1 分。

(3) 结合第六大题, 阐述以 Unix 为代表的分时系统, 多道程序分时共享 CPU 的具体实现机制。
(6 分)

时间片轮转, 4 分

非抢占调度和抢占调度, 2 分